

Contents

Tổng hợp truy vấn dữ liệu — ôn bảo vệ ĐATN	2
0. Kiến trúc truy vấn — phải nói được khi mở bài	2
1. Truy vấn CRUD đơn giản	2
1.1 Lấy thông tin một quân nhân kèm đơn vị, chức vụ	2
1.2 Tạo mới quân nhân kèm cập nhật bộ đếm đơn vị	3
1.3 Tìm trùng CCCD trước khi tạo quân nhân	3
2. Truy vấn có lọc + phân trang	3
2.1 Liệt kê quân nhân với bộ lọc	3
2.2 Lọc đề xuất theo trạng thái + năm + loại	4
3. Truy vấn aggregation (count, group by) cho dashboard	4
3.1 Đếm đề xuất theo trạng thái	4
3.2 Top 10 hành động hệ thống	5
3.3 Thống kê hoạt động 7 ngày gần nhất	5
4. Phân quyền theo cây đơn vị	5
4.1 Lấy phạm vi đơn vị của Manager	5
4.2 Liệt kê quân nhân Manager được phép xem	6
5. Truy vấn phức tạp nhất — Chuỗi danh hiệu hàng năm	6
5.1 Tải toàn bộ lịch sử danh hiệu của một quân nhân	6
5.2 Đếm số BKBQP trong cửa sổ trượt 3 năm (xét CSTĐTQ)	6
5.3 Upsert hồ sơ suy diễn HoSoHangNam	7
6. Transaction phức tạp — Phê duyệt đề xuất	7
6.1 Phê duyệt một đề xuất chứa N quân nhân	7
7. Nhật ký kiểm toán	8
7.1 Tạo log với payload before/after	8
7.2 Lọc nhật ký với cơ chế ẩn resource = 'backup'	9
8. Truy vấn analytic có thể demo trên psql lúc bảo vệ	9
8.1 Top 10 quân nhân được khen thưởng nhiều nhất	9
8.2 Tỷ lệ phê duyệt theo loại đề xuất	9
8.3 Quân nhân chưa nhận khen thưởng nào trong 5 năm gần nhất	10
8.4 Đơn vị có nhiều BKTTCN nhất	10
8.5 Hoạt động hệ thống theo giờ trong ngày	10
9. Câu hỏi giám khảo có thể hỏi và cách trả lời	10
Q1. “Tại sao em chọn Prisma thay vì viết SQL thuần?”	10
Q2. “Cấu trúc cây đơn vị có 2 cấp, lỡ sau cần nhiều cấp thì sao?”	10
Q3. “Tại sao bảng BangDeXuat lại có 4 cột JSON data_* thay vì tách bảng?”	10
Q4. “Làm sao em đảm bảo so_luong của đơn vị luôn khớp với số quân nhân thực?”	10
Q5. “Truy vấn nào em đánh giá là phức tạp nhất?”	11
Q6. “Em có dùng index gì không?”	11
Q7. “Có chống SQL Injection không?”	11
Q8. “Có chỗ nào em phải tối ưu performance đặc biệt không?”	11
10. Truy vấn nâng cao — top-N gần nhất, time window, trend	11
10.1 — 30 log gần nhất theo action APPROVE trong 30 ngày	11
10.2 — 30 đề xuất gần nhất theo trạng thái và năm	12
10.3 — Đề xuất kẹt ở PENDING quá 7 ngày (cảnh báo SLA)	12
10.4 — Hoạt động hệ thống theo ngày trong 30 ngày qua (trend)	12
10.5 — 30 quân nhân sắp đủ điều kiện BKBQP trong năm tới	13
10.6 — Đơn vị có tỷ lệ phê duyệt thấp nhất (cảnh báo)	13
10.7 — So sánh số khen thưởng năm hiện tại vs năm trước (year-over-year)	13
10.8 — Người dùng hoạt động nhiều nhất 7 ngày qua	13
10.9 — Quân nhân có nhiều bản ghi nhất bị thay đổi (đáng nghi)	14
11. Xử lý đồng thời, race condition và transaction	14
11.1 Tình huống race condition trong PM QLKT	14
11.2 Race condition khi cập nhật so_luong đơn vị	15
11.3 Transaction lồng + truyền tx qua nhiều Repository	15
11.4 Promise.all song song — cẩn thận khi 1 cái lỗi	15

11.5 Bulk import 500 dòng, dòng 250 lỗi → toàn bộ rollback	16
11.6 Deadlock và cách phòng tránh	16
11.7 Idempotency — phê duyệt nhiều lần cùng 1 đề xuất	17
11.8 Connection pool — backend 1 instance, ~50 user đồng thời	17
12. Câu hỏi giám khảo nâng cao + đáp án (kèm code)	17
Q9. “Hai Admin cùng phê duyệt một đề xuất ở cùng thời điểm — chuyện gì xảy ra?”	17
Q10. “Khi nhập 500 dòng Excel mà dòng 250 lỗi, dữ liệu 249 dòng đầu đã ghi thì sao?”	18
Q11. “Em chạy <code>Promise.all</code> song song 50 quân nhân để recalc — 1 cái lỗi thì sao?”	18
Q12. “Em có dùng <code>SELECT FOR UPDATE</code> bao giờ không?”	19
Q13. “Deadlock có xảy ra không và em chống thế nào?”	20
Q14. “Tại sao em chia bước Excel thành preview + confirm?”	20
Q15. “Idempotency — em xử lý replay request thế nào?”	21
Q16. “Em có dùng index không, làm sao biết khi nào cần thêm?”	22
Q17. “Connection pool em cấu hình ra sao? Tránh hết pool thế nào?”	22
Q18. “Tại sao em không dùng MongoDB hay NoSQL?”	23
Q19. “Em làm sao đảm bảo log kiểm toán không bị xóa hay sửa?”	24
Q20. “Race condition khi 2 user cùng tạo quân nhân với cùng CCCD?”	24
13. Cheat sheet — phải nhớ thuộc lòng	25

Tổng hợp truy vấn dữ liệu — ôn bảo vệ ĐATN

Tài liệu này tổng hợp các kiểu truy vấn dữ liệu tiêu biểu trong PM QLKT, sắp xếp từ dễ đến phức tạp. Mỗi mục gồm: (i) mô tả nghiệp vụ, (ii) cú pháp Prisma trong code, (iii) SQL tương đương, (iv) lý do thiết kế. Phần cuối là gợi ý các câu hỏi giám khảo có thể hỏi và cách trả lời.

0. Kiến trúc truy vấn — phải nói được khi mở bài

Toàn bộ truy vấn dữ liệu đi qua **Prisma ORM** chứ không viết SQL thuần. Lớp **Repository** đóng gói gọi Prisma; lớp **Service** chỉ điều phối logic nghiệp vụ. Khi cần bảo đảm nguyên tử tính, Service mở `prisma.$transaction(async tx => ...)` và truyền `tx` qua các Repository — đảm bảo tất cả query trong transaction cùng commit/rollback.

Schema thực thể chính: `QuanNhan` (quân nhân), `BangDeXuat` (đề xuất), `DanhHieuHangNam` (danh hiệu hằng năm), `HoSoHangNam / HoSoNienHan / HoSoCongHien` (hồ sơ suy diễn), `LichSuChucVu` (lịch sử chức vụ), `SystemLog` (nhật ký), `TaiKhoan` (tài khoản), `CoQuanDonVi / DonViTrucThuoc` (cây tổ chức 2 cấp).

1. Truy vấn CRUD đơn giản

1.1 Lấy thông tin một quân nhân kèm đơn vị, chức vụ

Mục đích: Hiển thị trang chi tiết quân nhân.

Prisma (`repositories/quanNhan.repository.ts`)

```
prisma.quanNhan.findUnique({
  where: { id: personnelId },
  include: { CoQuanDonVi: true, DonViTrucThuoc: true, ChucVu: true },
});
```

SQL tương đương

```
SELECT q.*, cq.ten_don_vi AS cqdv_ten, dvtt.ten_don_vi AS dvtt_ten, cv.ten_chuc_vu
FROM QuanNhan q
LEFT JOIN CoQuanDonVi cq ON q.co_quan_don_vi_id = cq.id
LEFT JOIN DonViTrucThuoc dv ON q.don_vi_truc_thuoc_id = dv.id
LEFT JOIN ChucVu cv ON q.chuc_vu_id = cv.id
WHERE q.id = $1;
```

Tại sao dùng `include` thay vì gọi 3 truy vấn riêng? Prisma sinh ra một câu `SELECT` có nhiều `LEFT JOIN` — chỉ một round-trip tới PostgreSQL, tránh N+1.

1.2 Tạo mới quân nhân kèm cập nhật bộ đếm đơn vị

Mục đích: Khi thêm quân nhân thì `so_luong` của đơn vị tăng 1.

Prisma (`services/personnel.service.ts`)

```
await prisma.$transaction(async (tx) => {
  const qn = await tx.quanNhan.create({ data });
  await tx.coQuanDonVi.update({
    where: { id: data.co_quan_don_vi_id },
    data: { so_luong: { increment: 1 } },
  });
  return qn;
});
```

SQL tương đương

```
BEGIN;
INSERT INTO QuanNhan (id, cccd, ho_ten, ngay_sinh, ...)
VALUES ($1, $2, ...);
UPDATE CoQuanDonVi SET so_luong = so_luong + 1 WHERE id = $3;
COMMIT;
```

Lý do: hai thao tác phải atomic — nếu một lỗi thì rollback cả hai để bộ đếm không bao giờ lệch với số bản ghi thực.

1.3 Tìm trùng CCCD trước khi tạo quân nhân

Prisma

```
prisma.quanNhan.findUnique({ where: { cccd } });
```

SQL tương đương

```
SELECT * FROM QuanNhan WHERE cccd = $1 LIMIT 1;
```

Note: Cột `cccd` có chỉ mục `UNIQUE` ở schema (`@unique` trong `schema.prisma`). PostgreSQL dùng B-tree index để tìm $O(\log n)$.

2. Truy vấn có lọc + phân trang

2.1 Liệt kê quân nhân với bộ lọc

Prisma

```
prisma.quanNhan.findMany({
  where: {
    AND: [
      filter.co_quan_don_vi_id ? { co_quan_don_vi_id: filter.co_quan_don_vi_id } : {},
      filter.gioi_tinh ? { gioi_tinh: filter.gioi_tinh } : {},
      filter.search ? { ho_ten: { contains: filter.search, mode: 'insensitive' } } : {}
    ],
  },
  skip: (page - 1) * limit,
  take: limit,
  orderBy: { createdAt: 'desc' },
```

```
include: { CoQuanDonVi: true, ChucVu: true },
});
```

SQL tương đương

```
SELECT q.*, cq.ten_don_vi, cv.ten_chuc_vu
FROM QuanNhan q
LEFT JOIN CoQuanDonVi cq ON q.co_quan_don_vi_id = cq.id
LEFT JOIN ChucVu cv ON q.chuc_vu_id = cv.id
WHERE ($1::text IS NULL OR q.co_quan_don_vi_id = $1)
AND ($2::text IS NULL OR q.gioi_tinh = $2)
AND ($3::text IS NULL OR q.ho_ten ILIKE '%' || $3 || '%')
ORDER BY q.createdAt DESC
OFFSET $4 LIMIT $5;
```

Note: mode: 'insensitive' ánh xạ sang ILIKE của PostgreSQL — không phân biệt hoa thường.

2.2 Lọc đề xuất theo trạng thái + năm + loại

Prisma

```
prisma.bangDeXuat.findMany({
  where: { status: 'PENDING', loai_de_xuat: 'CA_NHAN_HANG_NAM', nam: 2026 },
  include: { NguoiDeXuat: true, CoQuanDonVi: true },
  orderBy: { createdAt: 'desc' },
});
```

SQL tương đương

```
SELECT b.*, tk.username AS nguoi_de_xuat_username, cq.ten_don_vi
FROM BangDeXuat b
LEFT JOIN TaiKhoan tk ON b.nguoi_de_xuat_id = tk.id
LEFT JOIN CoQuanDonVi cq ON b.co_quan_don_vi_id = cq.id
WHERE b.status = 'PENDING'
AND b.loai_de_xuat = 'CA_NHAN_HANG_NAM'
AND b.nam = 2026
ORDER BY b.createdAt DESC;
```

3. Truy vấn aggregation (count, group by) cho dashboard

3.1 Đếm đề xuất theo trạng thái

Prisma (services/dashboard.service.ts)

```
prisma.bangDeXuat.groupBy({
  by: ['status'],
  _count: { _all: true },
  where: { nam: currentYear },
});
```

SQL tương đương

```
SELECT status, COUNT(*) AS so_luong
FROM BangDeXuat
WHERE nam = $1
GROUP BY status;
```

Kết quả: [{status:'PENDING', _count:12}, {status:'APPROVED', _count:47}, ...].

3.2 Top 10 hành động hệ thống

Prisma (repositories/systemLog.repository.ts)

```
prisma.systemLog.groupBy({
  by:      ['action'],
  _count:  { action: true },
  orderBy: { _count: { action: 'desc' } },
  take:    10,
});
```

SQL tương đương

```
SELECT action, COUNT(*) AS so_luong
FROM SystemLog
GROUP BY action
ORDER BY so_luong DESC
LIMIT 10;
```

3.3 Thống kê hoạt động 7 ngày gần nhất

Prisma

```
prisma.systemLog.findMany({
  where: { createdAt: { gte: sevenDaysAgo } },
  select: { createdAt: true },
});
// rồi group ở JS theo date
```

SQL tương đương (làm trên DB hiệu quả hơn)

```
SELECT DATE(createdAt) AS ngay, COUNT(*) AS so_luong
FROM SystemLog
WHERE createdAt >= NOW() - INTERVAL '7 days'
GROUP BY DATE(createdAt)
ORDER BY ngay;
```

Lý do: với dataset nhỏ (~vài nghìn log/ngày) group ở JS chấp nhận được. Với dataset lớn nên dùng \$queryRaw để aggregate ở DB.

4. Phân quyền theo cây đơn vị

4.1 Lấy phạm vi đơn vị của Manager

Prisma (middlewares/unitFilter.ts)

```
prisma.quanNhan.findUnique({
  where: { id: account.quan_nhan_id },
  select: { co_quan_don_vi_id: true, don_vi_truc_thuoc_id: true },
});
```

SQL tương đương

```
SELECT co_quan_don_vi_id, don_vi_truc_thuoc_id
FROM QuanNhan
WHERE id = $1;
```

4.2 Liệt kê quân nhân Manager được phép xem

Manager gắn với CQĐV → thấy tất cả quân nhân cùng CQĐV (kể cả thuộc DVTT con).

Prisma

```
prisma.quanNhan.findMany({
  where: {
    OR: [
      { co_quan_don_vi_id: managerCqdvId },
      { don_vi_truc_thuoc: { co_quan_don_vi_id: managerCqdvId } },
    ],
  },
});
```

SQL tương đương

```
SELECT q.*
FROM QuanNhan q
LEFT JOIN DonViTrucThuoc dv ON q.don_vi_truc_thuoc_id = dv.id
WHERE q.co_quan_don_vi_id = $1
      OR dv.co_quan_don_vi_id = $1;
```

Note: cấu trúc cây 2 cấp đơn giản, không phải đệ quy. Nếu mở rộng nhiều cấp sẽ cần CTE đệ quy WITH RECURSIVE.

5. Truy vấn phức tạp nhất — Chuỗi danh hiệu hàng năm

Đây là trọng tâm có thể bị hỏi nhiều nhất khi bảo vệ.

5.1 Tải toàn bộ lịch sử danh hiệu của một quân nhân

Prisma (services/profile/annual.ts)

```
prisma.danhHieuHangNam.findMany({
  where: { quan_nhan_id: personnelId },
  orderBy: { nam: 'desc' },
});
```

SQL tương đương

```
SELECT * FROM DanhHieuHangNam
WHERE quan_nhan_id = $1
ORDER BY nam DESC;
```

Lý do ORDER BY nam DESC: Hàm computeChainContext duyệt từ năm hiện tại lùi về quá khứ để tính độ dài chuỗi CSTĐCS liên tục, đếm cờ BKBQP/CSTĐTQ trong cửa sổ trượt 3 năm hoặc 7 năm.

5.2 Đếm số BKBQP trong cửa sổ trượt 3 năm (xét CSTĐTQ)

Logic kết hợp Prisma + JS:

```
// JS — sau khi load
const flagsInWindow = rows
  .filter(r => r.nam >= currentYear - 3 && r.nam < currentYear)
  .filter(r => r.nhan_bkbqp).length;
```

SQL tương đương (nếu chuyển hẳn xuống DB)

```
SELECT COUNT(*) FROM DanhHieuHangNam
WHERE quan_nhan_id = $1
```

```

AND nam >= $2 - 3
AND nam < $2
AND nhan_bkbqp = TRUE;

```

Lý do làm ở JS thay vì DB: lịch sử một quân nhân thường ≤ 30 dòng — load 1 lần rồi lọc nhiều lần ở JS nhanh hơn nhiều round-trip.

5.3 Upsert hồ sơ suy diễn HoSoHangNam

Sau khi tính xong, ghi kết quả vào bảng cache.

Prisma

```

prisma.hoSoHangNam.upsert({
  where: { quan_nhan_id: personnelId },
  create: { quan_nhan_id, cstdcs_lien_tuc, du_dieu_kien_bkbqp, du_dieu_kien_cstdtq, du_dieu_kien_bkttcp,
  update: { cstdcs_lien_tuc, du_dieu_kien_bkbqp, du_dieu_kien_cstdtq, du_dieu_kien_bkttcp,
});

```

SQL tương đương (PostgreSQL có cú pháp ON CONFLICT)

```

INSERT INTO HoSoHangNam (quan_nhan_id, cstdcs_lien_tuc, du_dieu_kien_bkbqp, ...)
VALUES ($1, $2, $3, ...)
ON CONFLICT (quan_nhan_id)
DO UPDATE SET
  cstdcs_lien_tuc      = EXCLUDED.cstdcs_lien_tuc,
  du_dieu_kien_bkbqp   = EXCLUDED.du_dieu_kien_bkbqp,
  ...
  last_recalc_at      = NOW();

```

Note: quan_nhan_id có ràng buộc UNIQUE (1-1 với QuanNhan) — vì là bảng suy diễn.

6. Transaction phức tạp — Phê duyệt đề xuất

6.1 Phê duyệt một đề xuất chứa N quân nhân

Khi Admin bấm “Phê duyệt”, một transaction duy nhất phải: 1. Đổi status đề xuất PENDING → APPROVED 2. Ghi N bản ghi vào DanhHieuHangNam (hoặc bảng tương ứng) 3. Gắn số quyết định + đường dẫn PDF 4. Ghi SystemLog với payload trước-sau 5. Tính lại HoSoHangNam cho N quân nhân bị ảnh hưởng

Prisma (services/proposal/approve/import.ts — đơn giản hoá)

```

await prisma.$transaction(async (tx) => {
  // (1) Chuyển trạng thái
  await tx.bangDeXuat.update({
    where: { id: proposalId },
    data: { status: 'APPROVED', approved_at: new Date() },
  });

  // (2) Ghi từng danh hiệu
  for (const item of proposal.data_danh_hieu) {
    await tx.danhHieuHangNam.upsert({
      where: { quan_nhan_id_nam: { quan_nhan_id: item.qn_id, nam: proposal.nam } },
      create: { ... },
      update: { ... },
    });
  }
}

```

```

// (3) Gắn quyết định
await tx.fileQuyetDinh.create({ data: { so_quyet_dinh, file_path, proposal_id: proposalId

// (4) Ghi log
await tx.systemLog.create({ data: { action: 'APPROVE', resource: 'proposals', payload: { b

// (5) Tính lại cho từng quân nhân
for (const qnId of affectedPersonnelIds) {
  await recalcAnnualProfile(tx, qnId);
}
});

```

SQL tương đương (giả lược — thực tế gồm vài chục câu trong 1 transaction):

```

BEGIN;

-- (1)
UPDATE BangDeXuat SET status = 'APPROVED', approved_at = NOW() WHERE id = $1;

-- (2) – N lần INSERT/UPDATE
INSERT INTO DanhHieuHangNam (...) VALUES (...) ON CONFLICT (...) DO UPDATE SET ...;

-- (3)
INSERT INTO FileQuyetDinh (so_quyet_dinh, file_path, proposal_id) VALUES (...);

-- (4)
INSERT INTO SystemLog (action, resource, payload) VALUES ('APPROVE', 'proposals', '{...}'::

-- (5) – N lần upsert HoSoHangNam
INSERT INTO HoSoHangNam (...) VALUES (...) ON CONFLICT (...) DO UPDATE SET ...;

COMMIT;

```

Lý do dùng transaction: Nếu dòng thứ 30 bị lỗi (vd: vi phạm rule lifetime BKTTCP), 29 dòng đã ghi trước phải rollback hết — không có cảnh “nửa duyệt nửa chưa”.

7. Nhật ký kiểm toán

7.1 Tạo log với payload before/after

Prisma (middleware auditLog.ts)

```

prisma.systemLog.create({
  data: {
    nguoi_thuc_hien_id: req.user.id,
    nguoi_thuc_hien_role: req.user.role,
    action: 'UPDATE',
    resource: 'personnel',
    resource_id: personnelId,
    description: `Cập nhật quân nhân ${ho_ten}`,
    payload: { before: oldData, after: newData }, // JSONB column
  },
});

```

SQL tương đương

```

INSERT INTO SystemLog (
  id, nguoi_thuc_hien_id, nguoi_thuc_hien_role,

```



```

    action, resource, resource_id, description, payload, createdAt
) VALUES ($1, $2, $3, $4, $5, $6, $7, $8::jsonb, NOW());

```

Note: Cột payload kiểu JSONB cho phép truy vấn theo trường con sau này, vd: WHERE payload->>'reason' = 'X'.

7.2 Lọc nhật ký với cơ chế ẩn resource = 'backup'

Prisma (services/systemLogs.service.ts)

```

const where = {
  AND: [
    filter.action ? { action: filter.action } : {},
    filter.from ? { createdAt: { gte: filter.from } } : {},
    // chỉ SuperAdmin thấy log backup
    userRole !== 'SUPER_ADMIN' ? { resource: { not: 'backup' } } : {},
    // Manager chỉ thấy log do account thuộc đơn vị mình
    userRole === 'MANAGER' ? { nguoi_thuc_hien_id: { in: managerAccountIds } } : {},
  ],
};
prisma.systemLog.findMany({ where, orderBy: { createdAt: 'desc' }, skip, take });

```

SQL tương đương

```

SELECT * FROM SystemLog
WHERE ($1::text IS NULL OR action = $1)
  AND ($2::timestampz IS NULL OR createdAt >= $2)
  AND ($3::text = 'SUPER_ADMIN' OR resource <> 'backup')
  AND ($3::text <> 'MANAGER' OR nguoi_thuc_hien_id = ANY($4))
ORDER BY createdAt DESC
OFFSET $5 LIMIT $6;

```

8. Truy vấn analytic có thể demo trên psql lúc bảo vệ

Những query này KHÔNG nằm trong code (chạy thử công khi demo) — show được khả năng đọc DB.

8.1 Top 10 quân nhân được khen thưởng nhiều nhất

```

SELECT q.ho_ten,
       COUNT(*) FILTER (WHERE d.nhan_bkbqp) AS so_bkbqp,
       COUNT(*) FILTER (WHERE d.nhan_cstdtq) AS so_cstdtq,
       COUNT(*) FILTER (WHERE d.nhan_bkttcp) AS so_bkttcp
FROM QuanNhan q
JOIN DanhHieuHangNam d ON d.quan_nhan_id = q.id
GROUP BY q.id, q.ho_ten
ORDER BY (
  COUNT(*) FILTER (WHERE d.nhan_bkbqp)
+ COUNT(*) FILTER (WHERE d.nhan_cstdtq)
+ COUNT(*) FILTER (WHERE d.nhan_bkttcp)
) DESC
LIMIT 10;

```

8.2 Tỷ lệ phê duyệt theo loại đề xuất

```

SELECT loai_de_xuat,
       COUNT(*) FILTER (WHERE status = 'APPROVED')::float / COUNT(*) AS ty_le_duyet
FROM BangDeXuat
GROUP BY loai_de_xuat

```

```
ORDER BY ty_le_duyet DESC;
```

8.3 Quân nhân chưa nhận khen thưởng nào trong 5 năm gần nhất

```
SELECT q.id, q.ho_ten
FROM QuanNhan q
WHERE NOT EXISTS (
  SELECT 1 FROM DanhHieuHangNam d
  WHERE d.quan_nhan_id = q.id
    AND d.nam >= EXTRACT(YEAR FROM CURRENT_DATE) - 5
);
```

8.4 Đơn vị có nhiều BKTTCP nhất

```
SELECT cq.ten_don_vi, COUNT(*) AS so_bkttcp
FROM QuanNhan q
JOIN DanhHieuHangNam d ON d.quan_nhan_id = q.id AND d.nhan_bkttcp = TRUE
JOIN CoQuanDonVi cq ON cq.id = q.co_quan_don_vi_id
GROUP BY cq.id, cq.ten_don_vi
ORDER BY so_bkttcp DESC
LIMIT 10;
```

8.5 Hoạt động hệ thống theo giờ trong ngày

```
SELECT EXTRACT(HOUR FROM createdAt) AS gio,
       COUNT(*) AS so_thao_tac
FROM SystemLog
WHERE createdAt >= NOW() - INTERVAL '30 days'
GROUP BY gio
ORDER BY gio;
```

9. Câu hỏi giám khảo có thể hỏi và cách trả lời

Q1. “Tại sao em chọn Prisma thay vì viết SQL thuần?”

Trả lời: (1) Prisma sinh kiểu TypeScript tự động từ `schema.prisma` nên mọi truy vấn có gợi ý ở thời điểm soạn thảo, giảm lỗi runtime; (2) Prisma sinh ra câu SQL tối ưu (đặc biệt với `include` — ghép JOIN trong 1 query thay vì N+1); (3) Migration tự động và có thể đảo ngược; (4) Khi cần performance cực cao, em vẫn có thể dùng `$queryRaw` (raw SQL) — nhưng thực tế chưa cần dùng trong dự án này.

Q2. “Cấu trúc cây đơn vị có 2 cấp, lỡ sau cần nhiều cấp thì sao?”

Trả lời: Hiện em mô hình hoá CQĐV (cha) — DVTT (con) chỉ 2 cấp vì đặc thù tổ chức của Học viện. Nếu mở rộng sâu hơn, em có thể chuyển sang mẫu **adjacency list** + **recursive CTE** của PostgreSQL: thêm cột `parent_id` self-reference, query bằng `WITH RECURSIVE`. Prisma chưa hỗ trợ recursive CTE trực tiếp nhưng có thể dùng `$queryRaw`.

Q3. “Tại sao bảng `BangDeXuat` lại có 4 cột JSON `data_*` thay vì tách bảng?”

Trả lời: Bảy loại đề xuất có schema chi tiết khác nhau (cá nhân hằng năm cần `data_danh_hieu`, niên hạn cần `data_nien_han`, công hiến cần `data_cong_hien`...). Nếu tách 7 bảng thì query “lấy tất cả đề xuất pending” phải UNION ALL 7 bảng — vừa rườm rà vừa chậm. Em chọn JSON để giữ schema linh hoạt; xác thực dữ liệu chi tiết được làm ở tầng Zod và `ProposalStrategy` của backend.

Q4. “Làm sao em đảm bảo `so_luong` của đơn vị luôn khớp với số quân nhân thực?”

Trả lời: Mọi thao tác thay đổi đơn vị của quân nhân (tạo, xoá, chuyển) đều nằm trong `prisma.$transaction` cùng với câu `update so_luong = so_luong + 1` (hoặc `-1`). Nếu một thao tác lỗi, transaction rollback cả hai → bộ đếm

không lệch. Ngoài ra em có script `recalcSoLuong.ts` ở DevZone để tính lại từ `COUNT(*)` thực — phòng trường hợp lỗi cũ làm lệch.

Q5. “Truy vấn nào em đánh giá là phức tạp nhất?”

Trả lời: Hàm `computeChainContext` xét chuỗi danh hiệu cá nhân. Phải duyệt lịch sử lùi từ năm hiện tại, tính (1) độ dài chuỗi CSTĐCS liên tục, (2) năm gần nhất nhận BKBQP/CSTĐTQ/BKTTCP, (3) số cờ BKBQP trong cửa sổ trượt 3 năm và 7 năm, (4) số CSTĐTQ trong cửa sổ 7 năm — tất cả để cấp đầu vào cho `checkChainEligibility`. Em chọn cách load 1 lần `DanhHieuHangNam` (thường ≤ 30 bản ghi/quân nhân) rồi xử lý ở JS thay vì 4-5 câu SQL `COUNT(*) WHERE` — vì round-trip DB tốn hơn nhiều.

Q6. “Em có dùng index gì không?”

Trả lời: PostgreSQL tự tạo B-tree index cho mọi cột PK và UNIQUE. Em đã thêm index thủ công cho các cột tra cứu thường xuyên qua migration `20260417_add_performance_indexes`: `BangDeXuat(status)`, `BangDeXuat(loai_de_xuat, nam)`, `SystemLog(createdAt)`, `DanhHieuHangNam(quan_nhan_id, nam)`, `LichSuChucVu(quan_nhan_id)`. Hai cột tổ hợp `quan_nhan_id + nam` ở `DanhHieuHangNam` là composite index đồng thời là UNIQUE.

Q7. “Có chống SQL Injection không?”

Trả lời: Có, chống mặc định ở 2 lớp: (1) **Zod validation** ở middleware kiểm tra kiểu/định dạng `req.body`, `req.query`, `req.params` — không hợp lệ thì trả 400 ngay; (2) **Prisma luôn dùng prepared statement** (parameterized query) — tham số được escape tự động, không thể inject SQL. Trường hợp `$queryRawUnsafe` (em chỉ dùng ở 1 script đổi tên cột) thì input là tên cột hard-code chứ không phải user input nên cũng an toàn.

Q8. “Có chỗ nào em phải tối ưu performance đặc biệt không?”

Trả lời: Có. Khi tính lại tổng thể (`POST /api/profile/recalc-all`) cho ~1.247 quân nhân, lần đầu em làm sequential mất ~50 giây. Sau khi: (1) batch load `DanhHieuHangNam` cho tất cả quân nhân trong 1 query (`where: {quan_nhan_id: {in: [...]}}`), (2) xử lý song song bằng `Promise.all` chia chunk 50 quân nhân, (3) batch `upsert HoSoHangNam` — thời gian giảm còn 18 giây.

10. Truy vấn nâng cao — top-N gần nhất, time window, trend

10.1 — 30 log gần nhất theo action `APPROVE` trong 30 ngày

Prisma

```
prisma.systemLog.findMany({
  where: {
    action: 'APPROVE',
    createdAt: { gte: new Date(Date.now() - 30 * 24 * 60 * 60 * 1000) },
  },
  orderBy: { createdAt: 'desc' },
  take: 30,
  include: { TaiKhoan: { include: { QuanNhan: { select: { ho_ten: true } } } } },
});
```

SQL

```
SELECT s.*, q.ho_ten AS nguoi_thuc_hien_ten
FROM SystemLog s
LEFT JOIN TaiKhoan tk ON s.nguoi_thuc_hien_id = tk.id
LEFT JOIN QuanNhan q ON tk.quan_nhan_id = q.id
WHERE s.action = 'APPROVE'
      AND s.createdAt >= NOW() - INTERVAL '30 days'
ORDER BY s.createdAt DESC
```

```
LIMIT 30;
```

10.2 — 30 đề xuất gần nhất theo trạng thái và năm

Prisma

```
prisma.bangDeXuat.findMany({
  where: { nam: 2026 },
  orderBy: [
    { status: 'asc' },          // PENDING < APPROVED < REJECTED theo alphabet
    { createdAt: 'desc' },
  ],
  take: 30,
  include: { NguoideXuat: true, CoQuanDonVi: true },
});
```

SQL

```
SELECT b.*
FROM BangDeXuat b
WHERE b.nam = 2026
ORDER BY b.status ASC, b.createdAt DESC
LIMIT 30;
```

10.3 — Đề xuất kẹt ở PENDING quá 7 ngày (cảnh báo SLA)

Prisma

```
prisma.bangDeXuat.findMany({
  where: {
    status: 'PENDING',
    createdAt: { lt: new Date(Date.now() - 7 * 24 * 60 * 60 * 1000) },
  },
  orderBy: { createdAt: 'asc' }, // cũ nhất trước
});
```

SQL

```
SELECT *, NOW() - createdAt AS thoi_gian_cho
FROM BangDeXuat
WHERE status = 'PENDING'
      AND createdAt < NOW() - INTERVAL '7 days'
ORDER BY createdAt ASC;
```

10.4 — Hoạt động hệ thống theo ngày trong 30 ngày qua (trend)

SQL (làm trên DB, hiệu quả cho trend chart)

```
SELECT DATE(createdAt) AS ngay,
       COUNT(*) FILTER (WHERE action = 'CREATE') AS tao_moi,
       COUNT(*) FILTER (WHERE action = 'UPDATE') AS cap_nhat,
       COUNT(*) FILTER (WHERE action = 'APPROVE') AS phe_duyet,
       COUNT(*) FILTER (WHERE action = 'DELETE') AS xoa,
       COUNT(*) AS tong
FROM SystemLog
WHERE createdAt >= NOW() - INTERVAL '30 days'
GROUP BY DATE(createdAt)
ORDER BY ngay;
```

10.5 — 30 quân nhân sắp đủ điều kiện BKBQP trong năm tới

SQL (kết hợp 2 điều kiện: chuỗi CSTĐCS hiện tại = 1, có NCKH năm hiện tại)

```
SELECT q.id, q.ho_ten, hsn.cstdcs_lien_tuc
FROM QuanNhan q
JOIN HoSoHangNam hsn ON hsn.quan_nhan_id = q.id
WHERE hsn.cstdcs_lien_tuc = 1 -- đang có 1 năm CSTĐCS
    AND EXISTS (
        SELECT 1 FROM DanhHieuHangNam d
        WHERE d.quan_nhan_id = q.id
            AND d.nam = EXTRACT(YEAR FROM CURRENT_DATE)
            AND d.danh_hieu = 'CSTDCS'
    )
ORDER BY q.ho_ten
LIMIT 30;
```

10.6 — Đơn vị có tỷ lệ phê duyệt thấp nhất (cảnh báo)

SQL

```
SELECT cq.ten_don_vi,
    COUNT(*) AS tong_de_xuat,
    COUNT(*) FILTER (WHERE b.status = 'APPROVED') AS so_duyet,
    ROUND(
        100.0 * COUNT(*) FILTER (WHERE b.status = 'APPROVED') / COUNT(*),
        2
    ) AS ti_le_duyet_pct
FROM BangDeXuat b
JOIN CoQuanDonVi cq ON cq.id = b.co_quan_don_vi_id
WHERE b.nam = 2026
GROUP BY cq.id, cq.ten_don_vi
HAVING COUNT(*) >= 5 -- bỏ đơn vị quá ít đề xuất
ORDER BY ti_le_duyet_pct ASC
LIMIT 10;
```

10.7 — So sánh số khen thưởng năm hiện tại vs năm trước (year-over-year)

SQL

```
WITH this_year AS (
    SELECT COUNT(*) AS sl FROM DanhHieuHangNam WHERE nam = 2026
),
last_year AS (
    SELECT COUNT(*) AS sl FROM DanhHieuHangNam WHERE nam = 2025
)
SELECT this_year.sl AS nam_2026,
    last_year.sl AS nam_2025,
    this_year.sl - last_year.sl AS chenh_lech,
    ROUND(100.0 * (this_year.sl - last_year.sl) / NULLIF(last_year.sl, 0), 2) AS pct_thay_doi
FROM this_year, last_year;
```

10.8 — Người dùng hoạt động nhiều nhất 7 ngày qua

SQL

```
SELECT tk.username, q.ho_ten, COUNT(*) AS so_thao_tac,
    MAX(s.createdAt) AS lan_cuoi
FROM SystemLog s
JOIN TaiKhoan tk ON s.nguoi_thuc_hien_id = tk.id
```

```
JOIN QuanNhan q ON tk.quan_nhan_id = q.id
WHERE s.createdAt >= NOW() - INTERVAL '7 days'
GROUP BY tk.id, tk.username, q.ho_ten
ORDER BY so_thao_tac DESC
LIMIT 20;
```

10.9 — Quân nhân có nhiều bản ghi nhất bị thay đổi (đáng nghi)

SQL (audit/security review)

```
SELECT s.resource_id AS quan_nhan_id, q.ho_ten, COUNT(*) AS so_lan_thay_doi
FROM SystemLog s
LEFT JOIN QuanNhan q ON q.id = s.resource_id
WHERE s.resource = 'personnel'
      AND s.action IN ('UPDATE', 'DELETE')
      AND s.createdAt >= NOW() - INTERVAL '30 days'
GROUP BY s.resource_id, q.ho_ten
HAVING COUNT(*) >= 5
ORDER BY so_lan_thay_doi DESC;
```

11. Xử lý đồng thời, race condition và transaction

11.1 Tình huống race condition trong PM QLKT

Vấn đề: Hai Admin A và B cùng mở chi tiết một đề xuất PENDING. A bấm “Phê duyệt” trước, sau đó B cũng bấm “Phê duyệt” mà không refresh — kết quả mong muốn: B phải nhận lỗi 409, KHÔNG được duyệt lần 2.

Giải pháp 1 — Optimistic locking dùng version field:

Thêm cột version vào BangDeXuat:

```
ALTER TABLE BangDeXuat ADD COLUMN version INT NOT NULL DEFAULT 0;
```

Khi update phê duyệt:

```
const updated = await prisma.bangDeXuat.updateMany({
  where: { id: proposalId, version: clientVersion, status: 'PENDING' },
  data: { status: 'APPROVED', version: { increment: 1 } },
});
if (updated.count === 0) {
  throw new ConflictError('Đề xuất đã bị thay đổi, vui lòng tải lại trang');
}
```

Giải thích: updateMany trả về count = 0 nếu WHERE không khớp (nghĩa là version đã thay đổi do A duyệt trước). Đây là cách kiểm tra “compare-and-swap” trên DB — atomic, không cần lock pessimistic.

Giải pháp 2 — Pessimistic lock (SELECT FOR UPDATE):

Dùng raw query trong transaction:

```
await prisma.$transaction(async tx => {
  const [proposal] = await tx.$queryRaw<Proposal[]>`
    SELECT * FROM BangDeXuat WHERE id = ${proposalId} FOR UPDATE
  `;
  if (proposal.status !== 'PENDING') {
    throw new ConflictError('Đề xuất đã được xử lý');
  }
  await tx.bangDeXuat.update({ where: { id: proposalId }, data: { status: 'APPROVED' } });
});
```

Khi nào dùng cái nào? - Optimistic (em đang dùng) khi conflict hiếm — hiệu năng tốt vì không lock. - **Pessimistic** (FOR UPDATE) khi conflict nhiều và logic dài — đảm bảo tuyệt đối nhưng giảm throughput.

11.2 Race condition khi cập nhật `so_luong` đơn vị

Vấn đề: 2 admin cùng tạo quân nhân vào cùng 1 đơn vị tại cùng thời điểm. Nếu code làm:

```
// SAI — race condition!
const dv = await prisma.coQuanDonVi.findUnique({where:{id}});
await prisma.coQuanDonVi.update({where:{id}, data:{so_luong: dv.so_luong + 1}});
```

Hai luồng đọc cùng `so_luong = 10`, mỗi luồng cộng 1 ghi 11 → kết quả 11 thay vì 12.

Giải pháp: dùng atomic increment của Prisma (sinh ra SET `so_luong = so_luong + 1` ở DB):

```
await prisma.coQuanDonVi.update({
  where: { id },
  data: { so_luong: { increment: 1 } },
});
```

PostgreSQL serialise mọi UPDATE trên cùng row — không thể xen ngang.

11.3 Transaction lồng + truyền `tx` qua nhiều Repository

Pattern em dùng:

```
// Repository nhận tham số tx tùy chọn
async function findByPersonnelId(
  personnelId: string,
  tx: PrismaLike = prisma, // PrismaLike = PrismaClient | Prisma.TransactionClient
) {
  return tx.danhHieuHangNam.findMany({ where: { quan_nhan_id: personnelId } });
}

// Service mở transaction rồi truyền tx
await prisma.$transaction(async (tx) => {
  const records = await danhHieuRepo.findByPersonnelId(personnelId, tx);
  await hoSoRepo.upsert(personnelId, computed, tx);
  await systemLogRepo.create(logData, tx);
});
```

Lý do: Tất cả query trong `$transaction` chia sẻ cùng connection và transaction context. Nếu bất kỳ câu nào throw → rollback hết. Nếu repo không nhận tx mà tự `prisma.x.findMany(...)`, query đó sẽ chạy ngoài transaction → không rollback được.

11.4 Promise.all song song — cẩn thận khi 1 cái lỗi

Vấn đề kinh điển:

```
// MISTAKE: 1 lỗi -> 4 cái còn lại đã chạy nhưng kết quả mất
await Promise.all([
  recalcAnnualProfile(qn1),
  recalcAnnualProfile(qn2),
  recalcAnnualProfile(qn3),
  recalcAnnualProfile(qn4),
  recalcAnnualProfile(qn5), // throw — Promise.all reject ngay
]);
```

Promise.all reject ngay khi promise đầu tiên fail; các promise kia VẪN đang chạy, kết quả chúng tạo ra (vd: ghi DB thành công) sẽ KHÔNG bị rollback nếu chúng tự mở transaction riêng.

Giải pháp 1 — bao toàn bộ trong 1 \$transaction ngoài cùng:

```
await prisma.$transaction(async (tx) => {
  await Promise.all(personnelIds.map(id => recalcAnnualProfile(id, tx)));
  // nếu 1 cái throw → tx tự rollback hết
});
```

Vì tất cả share cùng tx, khi rollback thì các thao tác đã ghi cũng được hoàn tác.

Giải pháp 2 — Promise.allSettled để biết cái nào fail:

```
const results = await Promise.allSettled(
  personnelIds.map(id => recalcAnnualProfile(id))
);
const failed = results
  .map((r, i) => r.status === 'rejected' ? { id: personnelIds[i], err: r.reason } : null)
  .filter(Boolean);
if (failed.length > 0) {
  await writeSystemLog({ action: 'RECALC_FAILED', payload: { failed } });
}
```

Khi nào dùng cái nào? - Recalc trong cùng đề xuất duyệt → **giải pháp 1** (cần atomic). - Recalc batch hằng ngày → **giải pháp 2** (chấp nhận 1 vài cái fail, ghi log để xử lý sau).

11.5 Bulk import 500 dòng, dòng 250 lỗi → toàn bộ rollback

Cài đặt (services/proposal/approve/import.ts):

```
async function confirmImport(rows: ImportRow[]) {
  return prisma.$transaction(async (tx) => {
    for (const [idx, row] of rows.entries()) {
      try {
        await tx.danhHieuHangNam.create({ data: row });
      } catch (e) {
        throw new ImportError(`Dòng ${idx + 1}: ${e.message}`, idx);
      }
    }
  }, {
    timeout: 60_000, // 60s — bulk lớn cần nhiều thời gian
    isolationLevel: 'Serializable',
  });
}
```

Quan sát hành vi: - Dòng 1-249 ghi thành công vào WAL của PostgreSQL. - Dòng 250 throw → Prisma rollback transaction. - PostgreSQL undo các thay đổi trong WAL → DB trở về trạng thái trước. - API trả 400 với chỉ số dòng lỗi cho user.

Lý do tách 2 bước “preview” + “confirm”: - Bước **preview** chỉ đọc & validate, không mở transaction → cho phép user fix file Excel trước khi tốn thời gian transaction dài. - Bước **confirm** mở transaction một lần với toàn bộ dòng đã valid → giảm rủi ro lỗi giữa chừng.

11.6 Deadlock và cách phòng tránh

Tình huống: Hai transaction T1 và T2 đều cập nhật QuanNhan A và QuanNhan B. - T1: lock A, đợi B - T2: lock B, đợi A → deadlock. PostgreSQL phát hiện sau ~1s và abort 1 transaction.

Cách phòng: 1. **Luôn lock theo cùng thứ tự** (vd: theo ID tăng dần). Trong transaction phê duyệt đề xuất, em sort `personnelIds.sort()` trước khi loop để mọi transaction đều duyệt các row theo cùng order. 2. **Tránh transaction quá dài** — chỉ những thao tác cần atomic mới đặt trong `$transaction`. Việc gửi email/socket.io đặt sau khi commit. 3. **Set timeout hợp lý:** `prisma.$transaction(fn, {timeout: 10_000})` — nếu transaction kéo dài bất thường thì abort sớm thay vì giữ lock vô hạn.

11.7 Idempotency — phê duyệt nhiều lần cùng 1 đề xuất

Vấn đề: Mạng chậm, FE gửi 2 request `POST /api/proposals/:id/approve` liên tiếp. Cần đảm bảo chỉ duyệt 1 lần.

Giải pháp: Backend kiểm tra `status === 'PENDING'` ngay đầu transaction:

```
await prisma.$transaction(async (tx) => {
  const proposal = await tx.bangDeXuat.findUnique({ where: { id }, select: { status: true } })
  if (!proposal) throw new NotFoundError();
  if (proposal.status !== 'PENDING') {
    throw new ConflictError('Đề xuất đã được xử lý');
  }
  // ... duyệt logic
});
```

Request thứ 2 thấy `status = 'APPROVED'` → trả 409, không duyệt lần 2.

11.8 Connection pool — backend 1 instance, ~50 user đồng thời

Cấu hình (Prisma đọc từ `DATABASE_URL`):

`DATABASE_URL=postgresql://user:pass@localhost:5432/qlkt?connection_limit=10&pool_timeout=20`

- `connection_limit=10` — pool tối đa 10 connection. Đủ cho ~50 user vì mỗi request HTTP thường giải phóng connection sau ~100ms.
- `pool_timeout=20` — request thứ 11 đợi tối đa 20s nếu pool hết connection.

Khi nào tăng pool? Khi log thấy nhiều `pool_timeout error` hoặc latency p95 cao hơn p50 nhiều — biểu hiện đang queue chờ connection.

12. Câu hỏi giám khảo nâng cao + đáp án (kèm code)

Q9. “Hai Admin cùng phê duyệt một đề xuất ở cùng thời điểm — chuyện gì xảy ra?”

Trả lời: Em xử lý bằng compare-and-swap (CAS): điều kiện `WHERE` chứa cả `status = 'PENDING'` nên chỉ 1 trong 2 request thành công.

Prisma

```
const updated = await prisma.bangDeXuat.updateMany({
  where: { id: proposalId, status: 'PENDING' }, // CAS condition
  data: { status: 'APPROVED', approved_at: new Date() },
});
if (updated.count === 0) {
  throw new ConflictError('Đề xuất đã được xử lý, vui lòng tải lại trang');
}
```

SQL

```
-- Admin A chạy trước:
UPDATE BangDeXuat SET status = 'APPROVED', approved_at = NOW()
```

```
WHERE id = $1 AND status = 'PENDING';
-- Postgres trả: UPDATE 1

-- Admin B chạy sau (status đã là APPROVED):
UPDATE BangDeXuat SET status = 'APPROVED', approved_at = NOW()
WHERE id = $1 AND status = 'PENDING';
-- Postgres trả: UPDATE 0 → backend throw 409
```

PostgreSQL serialise mọi UPDATE trên cùng row qua row-level lock, nên không có cảnh “cả 2 cùng update”.

Q10. “Khi nhập 500 dòng Excel mà dòng 250 lỗi, dữ liệu 249 dòng đầu đã ghi thì sao?”

Trả lời: Tất cả 500 dòng nằm trong 1 \$transaction. Khi dòng 250 throw, Prisma gửi ROLLBACK → 249 dòng trước cũng bị undo.

Prisma (services/proposal/approve/import.ts)

```
async function confirmImport(rows: ImportRow[]) {
  return prisma.$transaction(async (tx) => {
    for (let i = 0; i < rows.length; i++) {
      try {
        await tx.danhHieuHangNam.create({ data: rows[i] });
      } catch (e) {
        // Throw -> Prisma tự gửi ROLLBACK xuống Postgres
        throw new ImportError(`Dòng ${i + 1}: ${e.message}`, i + 1);
      }
    }
  }, { timeout: 60_000 });
}
```

SQL trong transaction

```
BEGIN;
INSERT INTO DanhHieuHangNam (...) VALUES (...); -- dòng 1   □
INSERT INTO DanhHieuHangNam (...) VALUES (...); -- dòng 2   □
...
INSERT INTO DanhHieuHangNam (...) VALUES (...); -- dòng 249 □
INSERT INTO DanhHieuHangNam (...) VALUES (...); -- dòng 250 □ (vd: trùng cccd)
ROLLBACK; -- Postgres undo 249 dòng từ WAL
```

Test verify (tests/import/personalAnnualImport.test.ts):

```
it('rollback toàn bộ khi dòng giữa lỗi', async () => {
  const before = await prisma.danhHieuHangNam.count();
  await expect(confirmImport(rowsWithError250)).rejects.toThrow(ImportError);
  const after = await prisma.danhHieuHangNam.count();
  expect(after).toBe(before); // không có dòng nào ghi
});
```

Q11. “Em chạy Promise.all song song 50 quân nhân để recalc — 1 cái lỗi thì sao?”

Trả lời: 2 mode tùy ngữ cảnh.

Mode A — cần atomic (recalc trong cùng phê duyệt):

```
await prisma.$transaction(async (tx) => {
  await Promise.all(
    personnelIds.map(id => recalcAnnualProfile(tx, id))
  )
});
```

```

);
// Nếu 1 promise throw, tx tự rollback toàn bộ
});

```

Tất cả 50 promise dùng chung tx → fail 1 = rollback 50.

Mode B — chấp nhận lỗi cục bộ (recalc batch hằng ngày):

```

const results = await Promise.allSettled(
  personnelIds.map(id => recalcAnnualProfile(prisma, id)) // mỗi promise có tx riêng
);
const failed = results
  .map((r, i) => r.status === 'rejected' ? { id: personnelIds[i], err: r.reason.message } : null)
  .filter(Boolean);

if (failed.length > 0) {
  await prisma.systemLog.create({
    data: { action: 'RECALC_FAILED', resource: 'profile', payload: { failed } },
  });
}

```

Anti-pattern phải tránh:

```

// SAI: Promise.all không bao trong $transaction
// 1 cái fail -> 49 cái thành công vẫn commit độc lập
await Promise.all(
  personnelIds.map(id => recalcAnnualProfile(prisma, id))
);

```

Q12. “Em có dùng **SELECT FOR UPDATE** bao giờ không?”

Trả lời: Hiện chưa cần (CAS đủ dùng), nhưng nếu cần lock pessimistic cho flow đọc-tính-ghi dài, em sẽ làm như sau:

Prisma raw query (Prisma 5 chưa có forUpdate API)

```

await prisma.$transaction(async (tx) => {
  // Lock row đề xuất, các transaction khác phải đợi
  const [proposal] = await tx.$queryRaw<BangDeXuat[]>`
    SELECT * FROM "BangDeXuat" WHERE id = ${proposalId} FOR UPDATE
  `;
  if (proposal.status !== 'PENDING') {
    throw new ConflictError('Đã được xử lý');
  }
  // Tính toán phức tạp ~3 giây ở đây, vẫn an toàn vì row đã lock
  const computed = await heavyEligibilityCheck(tx, proposal);
  await tx.bangDeXuat.update({
    where: { id: proposalId },
    data: { status: 'APPROVED', ...computed },
  });
});

```

SQL tương đương

```

BEGIN;
SELECT * FROM "BangDeXuat" WHERE id = $1 FOR UPDATE;
-- ... computation ...
UPDATE "BangDeXuat" SET status = 'APPROVED' WHERE id = $1;
COMMIT;

```

Khi nào dùng FOR UPDATE thay vì CAS: khi đọc nhiều cột để tính rồi mới update — CAS chỉ tốt khi check 1 cở (status=PENDING).

Q13. “Deadlock có xảy ra không và em chống thế nào?”

Trả lời: Em chống bằng 2 kỹ thuật.

Kỹ thuật 1 — Lock theo thứ tự nhất quán:

```
// SAI: thứ tự update phụ thuộc thứ tự input -> dễ deadlock
await Promise.all(personnelIds.map(id => updatePersonnel(tx, id)));

// ĐÚNG: sort trước -> mọi transaction đung row theo cùng order
const sortedIds = [...personnelIds].sort();
for (const id of sortedIds) {
  await updatePersonnel(tx, id);
}
```

Kỹ thuật 2 — Timeout transaction:

```
await prisma.$transaction(async (tx) => {
  // ... heavy work ...
}, {
  timeout: 10_000,          // 10s - abort nếu kéo dài bất thường
  maxWait: 5_000,          // 5s - đợi lock tối đa
});
```

Khi Postgres detect deadlock (~1s sau khi cycle):

```
ERROR: deadlock detected
DETAIL: Process 12345 waits for ShareLock on transaction 67890;
       blocked by process 23456.
HINT: See server log for query details.
```

Postgres tự huỷ 1 transaction (transaction “victim”) và return lỗi 40P01. Backend bắt và retry tối đa 3 lần với exponential backoff.

Q14. “Tại sao em chia bước Excel thành preview + confirm?”

Trả lời: Để giảm thời gian giữ transaction lock.

Bước preview — chỉ đọc, KHÔNG mở transaction:

```
// services/proposal/strategies/caNhanHangNamStrategy.ts
async previewImport(rows: ImportRow[]) {
  const valid: ImportRow[] = [];
  const errors: ErrorRow[] = [];
  for (let i = 0; i < rows.length; i++) {
    const result = schema.validate(rows[i]);
    if (result.error) {
      errors.push({ row: i + 1, message: result.error.message });
    } else {
      // Read-only check: CCCD tồn tại?
      const exists = await prisma.quanNhan.findUnique({
        where: { cccd: rows[i].cccd },
        select: { id: true },
      });
      if (!exists) errors.push({ row: i + 1, message: 'CCCD không tồn tại' });
    }
  }
}
```

```

        else valid.push(rows[i]);
    }
}
return { valid, errors };
}

```

Bước confirm — mở transaction CHỈ với dòng đã valid:

```

async confirmImport(validRows: ImportRow[]) {
    return prisma.$transaction(async (tx) => {
        for (const row of validRows) {
            await tx.danhHieuHangNam.create({ data: row });
        }
    }, { timeout: 60_000 });
}

```

Lợi ích do được: với 500 dòng + 50 dòng lỗi, nếu làm 1 bước thì transaction giữ lock 30s → rollback toàn bộ → user phải sửa rồi chạy lại 30s nữa. Tách 2 bước: preview 5s không lock, sau khi sửa file confirm chỉ 3s với 450 dòng valid.

Q15. “Idempotency — em xử lý replay request thế nào?”

Trả lời: Mọi endpoint mutate kiểm state đầu transaction.

Prisma

```

async function approveProposal(proposalId: string, userId: string) {
    return prisma.$transaction(async (tx) => {
        const proposal = await tx.bangDeXuat.findUnique({
            where: { id: proposalId },
            select: { status: true },
        });
        if (!proposal) throw new NotFoundError('Không tìm thấy đề xuất');
        if (proposal.status !== 'PENDING') {
            throw new ConflictError('Đề xuất đã được xử lý');
        }
        // ... approve logic
    });
}

```

SQL

```

BEGIN;
SELECT status FROM "BangDeXuat" WHERE id = $1;
-- Nếu status != 'PENDING' -> rollback + throw 409
-- Nếu status == 'PENDING' -> tiếp tục approve
UPDATE "BangDeXuat" SET status = 'APPROVED' WHERE id = $1 AND status = 'PENDING';
COMMIT;

```

Detect replay từ log:

```

-- Tìm request duplicate (2 APPROVE cùng resource trong < 1 giây)
SELECT resource_id, COUNT(*), MIN(createdAt), MAX(createdAt)
FROM SystemLog
WHERE action = 'APPROVE'
      AND createdAt >= NOW() - INTERVAL '1 day'
GROUP BY resource_id
HAVING COUNT(*) > 1
      AND EXTRACT(EPOCH FROM MAX(createdAt) - MIN(createdAt)) < 1;

```

Q16. “Em có dùng index không, làm sao biết khi nào cần thêm?”

Trả lời: Có, đã thêm trong migration 20260417_add_performance_indexes.

Schema Prisma

```
model BangDeXuat {
  // ...
  @@index([status, nam])           // composite index
  @@index([loai_de_xuat])
  @@index([nguoi_de_xuat_id])
}

model SystemLog {
  // ...
  @@index([createdAt(sort: Desc)]) // dùng cho ORDER BY DESC
  @@index([resource, createdAt])
  @@index([nguoi_thuc_hien_id])
}

model DanhHieuHangNam {
  // ...
  @@unique([quan_nhan_id, nam])    // composite UNIQUE = composite index
}
```

SQL DDL tương đương (sinh trong migration.sql)

```
CREATE INDEX "BangDeXuat_status_nam_idx"
ON "BangDeXuat" (status, nam);

CREATE INDEX "SystemLog_createdAt_desc_idx"
ON "SystemLog" (createdAt DESC);

CREATE UNIQUE INDEX "DanhHieuHangNam_quan_nhan_id_nam_key"
ON "DanhHieuHangNam" (quan_nhan_id, nam);
```

Phát hiện query thiếu index bằng EXPLAIN:

```
EXPLAIN ANALYZE
SELECT * FROM SystemLog
WHERE resource = 'personnel'
AND createdAt >= NOW() - INTERVAL '7 days'
ORDER BY createdAt DESC LIMIT 30;
```

Output mong đợi:

Index Scan using SystemLog_resource_createdAt_idx ... (đúng)

Nếu thấy Seq Scan trên bảng > 10k row, là dấu hiệu cần thêm index.

Q17. “Connection pool em cấu hình ra sao? Tránh hết pool thế nào?”

Trả lời: Pool mặc định 10 connection, đủ ~50 user đồng thời.

Cấu hình (.env)

```
DATABASE_URL="postgresql://user:pass@localhost:5432/qlkt?connection_limit=10&pool_timeout=10"
```

Pattern release connection sớm:

```
// SAI: pg_dump (~30s) chạy trong $transaction -> giữ connection lock
await prisma.$transaction(async (tx) => {
```

```

const data = await tx.quanNhan.findMany();
await spawnPgDump('backup.sql'); // □ I/O dài giữ tx
await tx.systemLog.create({ data: ... });
});

// ĐÚNG: I/O nặng tách ngoài transaction
const data = await prisma.quanNhan.findMany();
await spawnPgDump('backup.sql'); // □ không giữ connection
await prisma.systemLog.create({ data: ... });

```

Monitor pool exhaustion:

```

-- Check Postgres bao nhiêu connection từ backend đang active
SELECT pid, username, application_name, state, query_start, state_change
FROM pg_stat_activity
WHERE application_name LIKE 'prisma%'
ORDER BY state_change;

```

Nếu thấy nhiều state idle in transaction lâu — đang có transaction chạy quá dài → cần optimize.

Q18. “Tại sao em không dùng MongoDB hay NoSQL?”

Trả lời: PostgreSQL hỗ trợ JSONB, vẫn được linh hoạt schema mà không mất ACID + relational.

Schema Prisma (cột JSONB)

```

model BangDeXuat {
  id          String    @id
  loai_de_xuat String
  data_danh_hieu Json?   // JSONB column
  data_thanh_tich Json?
  data_nien_han  Json?
  data_cong_hien Json?
}

```

Truy vấn JSONB như NoSQL:

```

// Prisma JSON path filter
prisma.bangDeXuat.findMany({
  where: {
    data_danh_hieu: {
      path: ['ranking'],
      equals: 'gold',
    },
  },
});

```

SQL tương đương (PostgreSQL JSONB operator):

```

SELECT * FROM "BangDeXuat"
WHERE data_danh_hieu->>'ranking' = 'gold';

```

-- Hoặc with index trên JSONB key:

```

CREATE INDEX idx_dh_ranking ON "BangDeXuat" USING gin ((data_danh_hieu->>'ranking'));

```

MongoDB tương đương (chỉ minh họa — em không dùng):

```

db.proposals.find({ "data_danh_hieu.ranking": "gold" });

```

PostgreSQL có cả 2 ưu điểm: ACID + relational + JSONB linh hoạt như MongoDB.

Q19. “Em làm sao đảm bảo log kiểm toán không bị xoá hay sửa?”

Trả lời: 3 lớp bảo vệ.

Lớp 1 — Code chỉ INSERT, không có route UPDATE:

```
// repositories/systemLog.repository.ts – chỉ có create
export const systemLogRepository = {
  create: (data, tx = prisma) => tx.systemLog.create({ data }),
  findMany: (where, tx = prisma) => tx.systemLog.findMany(where),
  cleanup: (cutoff, tx = prisma) => tx.systemLog.deleteMany({
    where: { createdAt: { lt: cutoff } },
  }),
  // KHÔNG có update / hardDelete
};
```

Lớp 2 — Database REVOKE quyền UPDATE/DELETE cho user backend:

```
-- Tạo role riêng cho backend
CREATE USER qlkt_app WITH PASSWORD '...';
GRANT SELECT, INSERT ON SystemLog TO qlkt_app;
-- KHÔNG grant UPDATE/DELETE
GRANT DELETE ON SystemLog TO qlkt_admin; -- chỉ admin DBA mới xoá được
```

Lớp 3 — Partition theo tháng + READ-ONLY partition cũ:

```
-- Convert SystemLog thành partitioned table theo createdAt
CREATE TABLE SystemLog_2026_05 PARTITION OF SystemLog
  FOR VALUES FROM ('2026-05-01') TO ('2026-06-01');

-- Sau khi tháng kết thúc, set partition cũ READ-ONLY
ALTER TABLE SystemLog_2026_04 SET (autovacuum_enabled = false);
REVOKE INSERT, UPDATE, DELETE ON SystemLog_2026_04 FROM PUBLIC;
```

Hiện em mới làm Lớp 1; Lớp 2 và 3 là hướng phát triển ghi tại Chương 6.

Q20. “Race condition khi 2 user cùng tạo quân nhân với cùng CCCD?”

Trả lời: Postgres UNIQUE constraint chặn ở lớp DB, không cần lock.

Schema Prisma

```
model QuanNhan {
  id String @id @default(cuid())
  cccd String @unique // <-- UNIQUE constraint
  // ...
}
```

SQL DDL

```
CREATE UNIQUE INDEX "QuanNhan_cccd_key" ON "QuanNhan"(cccd);
```

Behavior: 2 INSERT đồng thời với cùng cccd:

```
-- T1 (trước): INSERT INTO QuanNhan (cccd, ...) VALUES ('123', ...); -- □
-- T2 (cùng lúc): INSERT INTO QuanNhan (cccd, ...) VALUES ('123', ...);
-- ERROR: duplicate key value violates unique constraint "QuanNhan_cccd_key"
-- DETAIL: Key (cccd)=(123) already exists.
-- SQLSTATE: 23505
```


Backend bắt lỗi P2002 của Prisma:

```
try {
  await prisma.quanNhan.create({ data });
} catch (e) {
  if (e instanceof Prisma.PrismaClientKnownRequestError && e.code === 'P2002') {
    throw new ConflictError('Số CCCD đã tồn tại trong hệ thống');
  }
  throw e;
}
```

Test verify (tests/concurrent/createPersonnel.test.ts):

```
it('chỉ 1 trong 2 INSERT đồng thời thành công', async () => {
  const data = { cccd: '123456789012', ho_ten: 'Test' };
  const results = await Promise.allSettled([
    createPersonnel(data),
    createPersonnel(data),
  ]);
  const fulfilled = results.filter(r => r.status === 'fulfilled');
  const rejected = results.filter(r => r.status === 'rejected');
  expect(fulfilled.length).toBe(1);
  expect(rejected.length).toBe(1);
  expect(rejected[0].reason).toBeInstanceOf(ConflictError);
});
```

13. Cheat sheet — phải nhớ thuộc lòng

Khái niệm	Cú pháp Prisma	SQL
Lấy 1 bản ghi	<code>findUnique({where:{id}})</code>	<code>SELECT ... WHERE id = \$1</code>
Lọc nhiều điều kiện	<code>findMany({where:{AND:[...]}}</code>	<code>SELECT ... WHERE a AND b</code>
OR	<code>where:{OR:[{a},{b}]}</code>	<code>WHERE a OR b</code>
LIKE	<code>contains: 's', mode:'insensitive'</code>	<code>ILIKE '%s%'</code>
JOIN	<code>include:{Rel:true}</code>	<code>LEFT JOIN ... ON ...</code>
Phân trang	<code>skip, take</code>	<code>OFFSET, LIMIT</code>
Đếm	<code>count({where})</code>	<code>SELECT COUNT(*)</code>
Group by	<code>groupBy({by:[col], _count:{}})</code>	<code>GROUP BY</code>
Upsert	<code>upsert({where, create, update})</code>	<code>INSERT ... ON CONFLICT DO UPDATE</code>
Transaction	<code>\$transaction(async tx => ...)</code>	<code>BEGIN; ... COMMIT;</code>
Tăng/giảm	<code>data:{so_luong:{increment:1}}</code>	<code>SET so_luong = so_luong + 1</code>
In list	<code>where:{id:{in:[...]}}</code>	<code>WHERE id IN (...)</code>
Date range	<code>createdAt:{gte:from, lt:to}</code>	<code>WHERE createdAt >= \$1 AND createdAt < \$2</code>
Sắp xếp nhiều cột	<code>orderBy:[{a:'desc'},{b:'asc'}]</code>	<code>ORDER BY a DESC, b ASC</code>